

LLD Case Study 11: Automated Vending Machine

Target Patterns: State Pattern, State Machine Transitions

Key Engineering Focus: Hardware lifecycle modeling, coin balance tracking, inventory deduction, and change calculation.

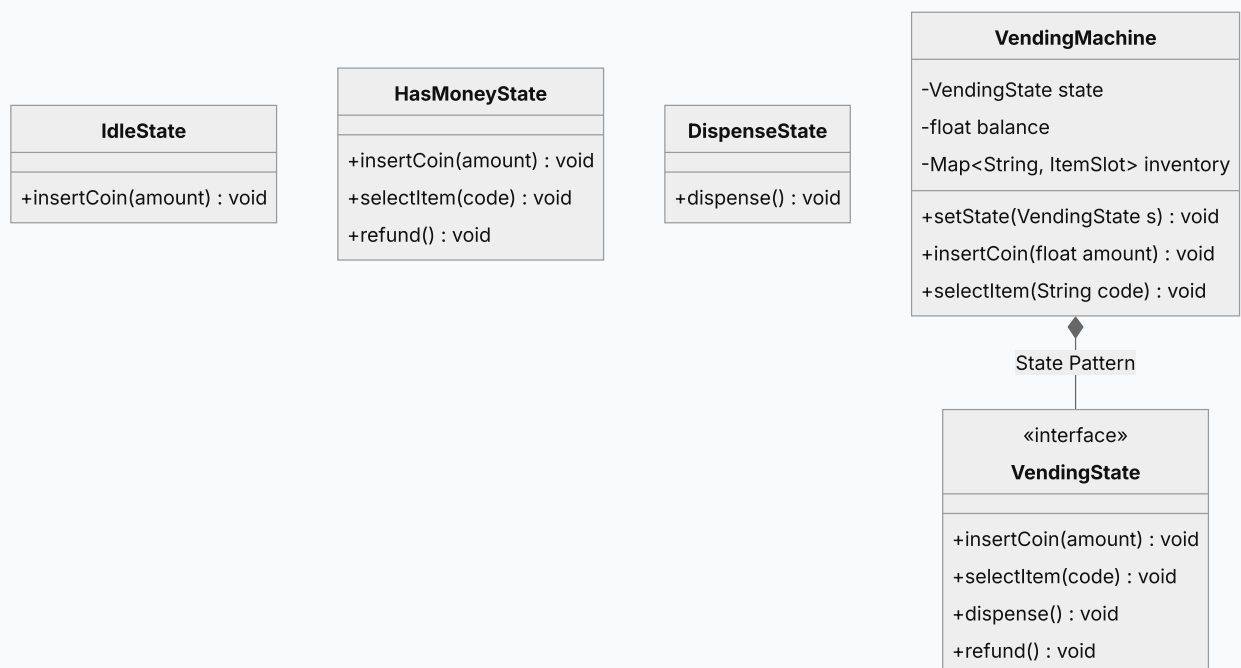
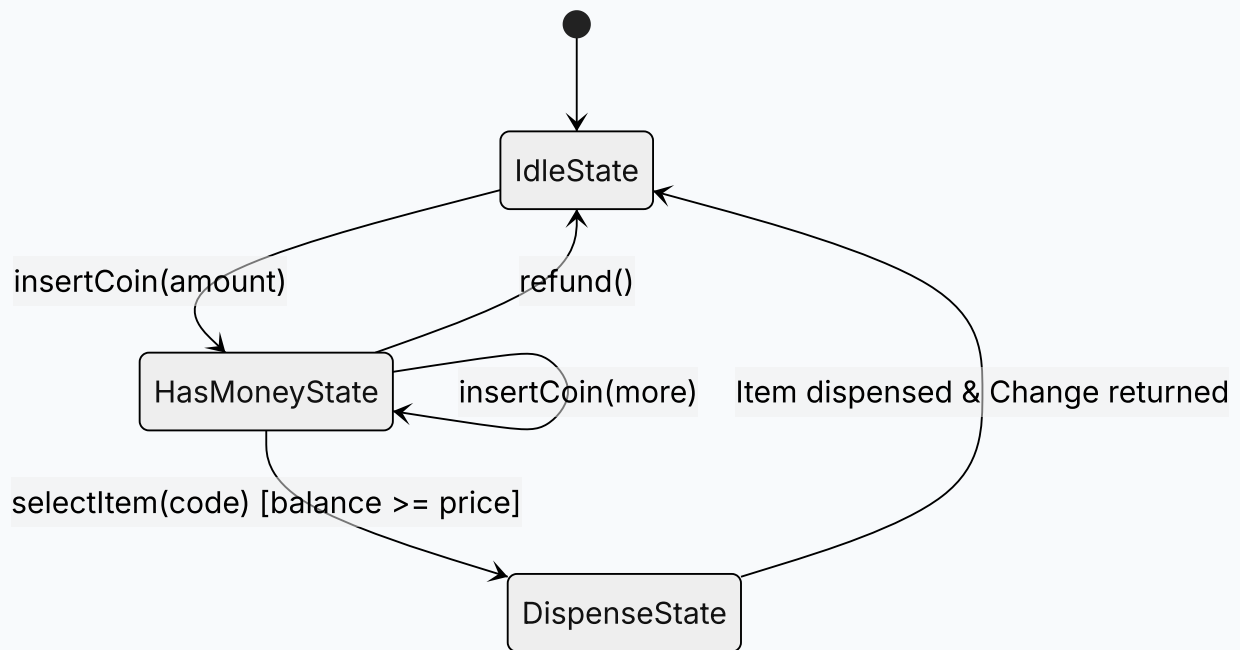
1. Problem Statement & Functional Requirements

Design an automated snack and beverage vending machine.

Requirements:

1. **Item Catalog & Inventory:** Slots storing items with fixed prices and quantities.
 2. **State Pattern:** The machine transitions through strict states:
 - **IDLE** : Awaiting money insertion.
 - **HAS_MONEY** : Tracking user balance; accepts further coins or item selection.
 - **DISPENSING** : Deducting inventory and dispensing item.
 - **SOLD_OUT** : Out of stock state.
 3. **Change Return:** Computes exact change if user inserts more than the item price.
-

2. Architecture & State Diagram



3. Production-Grade Python Implementation

4. Edge Cases, Tests and Extensions

State-machine review

State	<code>insert_coin</code>	<code>select_item</code>	<code>refund</code>	Failure to watch for
Idle	to HasMoney	prompt only	returns 0	none
HasMoney	adds balance	dispense, sold out, or short of funds	returns balance, to Idle	float balance can look short by a tiny amount
Dispense	rejected	rejected	rejected	power loss mid-dispense: persist the transaction first

Every illegal operation in a state is handled by that state class, so there is no `if state = ...` ladder in the context object. That is the point of the State pattern, and it is the property to point out in an interview.

Two weaknesses: money is stored as `float` dollars, and the machine returns "change" as a number without checking that it holds the coins to pay it.

Tests

This block extends the implementation above. The bug test shows the float problem; the next test shows that the same machine is exact when the amounts are integer cents.

Extensions interviewers ask for

- Make change from a coin inventory:** add a `CoinBank` with counts per denomination and a greedy or dynamic-programming change-maker; refuse the sale (or demand exact change) when change cannot be formed. Greedy is correct for canonical coin systems such as US coins and wrong for arbitrary ones, so say which you assume.
- Timeouts:** an unselected balance should refund after 30 seconds; give the machine a clock and a `tick()` that moves HasMoney to Idle, tested with a fake clock like the rate limiter.
- Payment by card:** a `PaymentStrategy` (cash, card, mobile) so `HasMoneyState` stops assuming coins.
- Remote monitoring:** emit events (`ItemSold` , `LowStock` , `Jammed`) to observers, which is the same Observer shape used in the inventory case study.

Follow-up questions

- *State pattern or an enum with a switch?* A switch is fine for three states and one method; the pattern pays off when states multiply, because each new state is a new class and existing ones stay untouched.
- *What if the machine loses power while dispensing?* Write the sale intent to durable storage before moving the motor, and on boot reconcile: either complete the dispense or refund.
- *Where would you add a maintenance mode?* A fourth state (`OutOfServiceState`) that rejects coins and allows only restocking commands.